MoA Prediction

A Multi-Model Ensemble for Mechanisms-of-Action Classification

Marco Merlonghi | Project Work in Machine Learning and Data Mining

About the Competition

- **Host:** the Broad Institute of MIT and Harvard, the Laboratory for Innovation Science at Harvard (LISH), and the NIH Common Funds Library of Integrated Network-Based Cellular Signatures (LINCS);
- **Platform:** Kaggle – `https://kaggle.com/c/lish-moa`;
- **Timeline:** launched and closed in 2020 (final deadline Nov. 30, 2020);
- **Scale:** 4,373 teams on the final leaderboard;
- **Format:** code competition – a notebook is submitted and re-run by Kaggle on a hidden test set;
- **Data origin:** gene expression from the L1000 assay, which measures how human cell lines respond transcriptionally to a chemical perturbation; cell viability from the PRISM assay.

Why it matters

Mapping a compound's gene-expression and cell-viability signature to its mechanism of action accelerates **drug discovery** and **repurposing**, replacing slow, costly wet-lab characterisation with a computational first pass.

The Problem and the Data

- **Task:** multi-label classification of **206 mechanisms of action (MoA)**
- **Input:** 772 gene-expression (`g-`) and 100 cell-viability (`c-`) features, plus `cp_time` and `cp_dose`
- **Size:** 23,814 training rows: 21,948 treated and 1,866 controls (7.8%)
- **Imbalance:** only 0.373% of the (treated row, target) cells are positive
- **Metric:** mean multi-label binary log loss, clipped to avoid log(0)
- Control rows (`cp_type == ctl_vehicle`) carry **no MoA**: excluded from training, set to 0 at submission

Why an ensemble?

Blending different model families (dense net, attention, trees) fed with different preprocessing is a standard way to reduce variance. In this project, both the blending gain and the diversity of the models' errors were evaluated.

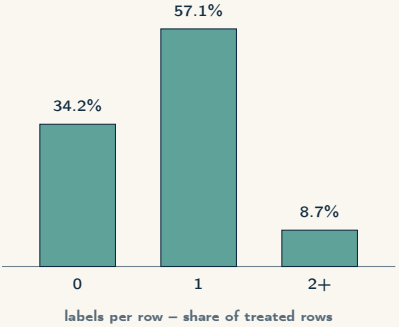
Exploratory Data Analysis

- **Controls:** 1,866 rows, **0 positive labels** – predicting 0 for them is safe
- **Label cardinality** (treated rows): 34.2% have no label, 57.1% one, 8.7% two or more (max 7)
- **Target imbalance:** positives per target range from 1 to 832 (median 38); **53 of 206 targets** have fewer than 20 positives
- **Design:** `cp_time` 24/48/72 h (7,166 / 7,602 / 7,180 treated rows), `cp_dose` D1/D2 (11,196 / 10,752)

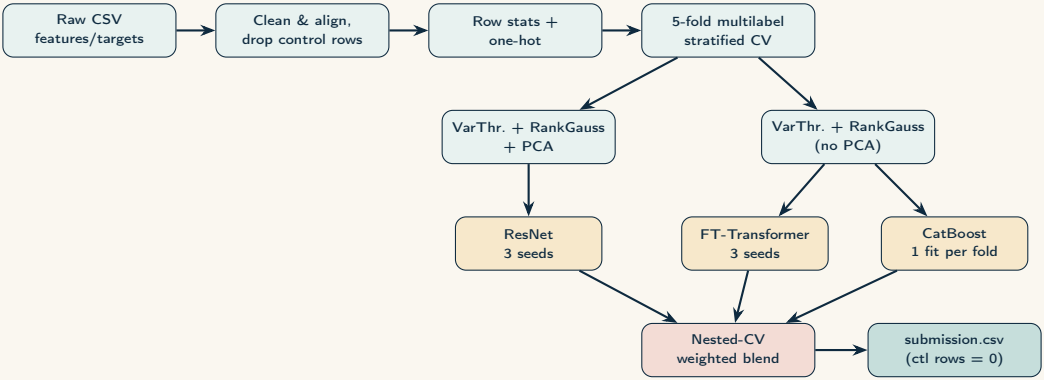
Consequences for the design

Rare targets dominate the difficulty ⇒ **multilabel-stratified** folds, so every target keeps its positives in each fold; controls are dropped from training.

Positive labels per treated row



Pipeline at a Glance



Two Preprocessing Pipelines

PCA variant (ResNet)

VarianceThreshold → RankGauss (`QuantileTransformer`) → PCA(50) on `g-`, PCA(15) on `c-` → **concatenate** transformed features, PCA components and raw dummies (≈950 columns)

Raw variant (FT-Transformer, CatBoost)

VarianceThreshold → RankGauss, **no PCA** – keeps one token/split per original feature (≈890 columns)

Both fitted **per fold** on the training split only and cached to disk as `.joblib`. The raw pipeline fitted for the transformer is reused by CatBoost within the same run; stale caches are cleared at the start of each run. The fitted pipelines are saved as artifacts, so the inference notebook only applies them (no refitting).

Cleaning, Feature Engineering and Hyperparameters

Cleaning and alignment

- `clean_and_align`: drops NaNs/duplicates, keeps the `sig_id`s present in both tables, sorts by `sig_id`
- Control rows dropped from training
- **Strict alignment check** before proceeding:

```
assert features.shape[0] == targets.shape[0]
assert (features["sig_id"].values
      == targets["sig_id"].values).all()
```

Feature engineering

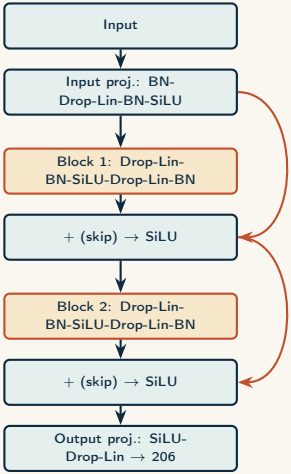
- Row sum, mean, std, kurtosis, skewness, separately for the `g-` and `c-` blocks
- One-hot on `cp_time` and `cp_dose`; test columns realigned to the training columns

Main hyperparameters

```
N_FOLDS = 5
SEEDS = [42, 202, 2024]
EPOCHS = 35 # AdamW, lr 1e-3
WEIGHT_DECAY = 1e-5
EARLY_STOPPING_PATIENCE = 8
INNER_VAL_FRACTION = 0.1
LABEL_SMOOTHING = 0.001
PCA_G_COMPONENTS = 50
PCA_C_COMPONENTS = 15
CATBOOST: 250 iters, depth 5,
       lr 0.08
```

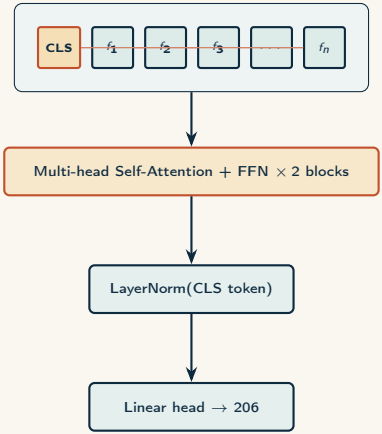
- `iterative-stratification` for multilabel stratified K-fold
- AMP enabled only if a GPU is present

TabularResNet – How It Works



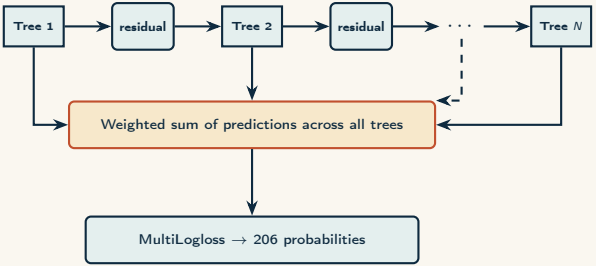
- The curved orange arrows are the **residual (skip) connections**: the block's input is added to its output
- Eases gradient flow, stabilises deeper networks
- 2 residual blocks, `hidden_dim = 512`, dropout 0.2
- Consumes **RankGauss + PCA** features: the transformed features *plus* 50 + 15 PCA components (≈950 inputs)

FT-Transformer – How It Works



- Each scalar feature → its own **token** (≈890 tokens/row incl. CLS)
- A learnable **[CLS]** token is prepended
- **Self-attention** (thin lines) connects every token to every other
- The CLS output summarises the row → classification head
- Token size 64, 2 blocks, 4 heads, dropout 0.2
- $O(L^2)$ cost ⇒ smaller batch size (64)
- Consumes **raw RankGauss** features (no PCA)
- Simplified variant of Gorishniy et al. (post-LayerNorm, GELU, no ReGLU)

CatBoost – How It Works



- **Gradient boosting**: each tree corrects the residual error of the previous ones
- A single multi-output `MultiLogloss` fit, not 206 separate fits
- 250 symmetric trees of depth 5, learning rate 0.08
- Splits stay aligned to the original features → trained on **raw RankGauss** (no PCA)
- Fixed iterations, one fit per fold, no seed bagging
- Targets constant within a fold are excluded from the fit and predicted with a **smoothed prior** (0.5 pseudo-counts) instead of 0

Metric, Loss and Training Routine

Evaluation metric

$$\mathcal{L} = -\frac{1}{N \cdot 206} \sum_{i=1}^{N} \sum_{j=1}^{206} \Big[y_{ij} \log \hat{y}_{ij} + (1 - y_{ij}) \log(1 - \hat{y}_{ij}) \Big]$$

predictions clipped to $[\epsilon, 1-\epsilon]$, with $\epsilon = 10^{-15}$, computed in float64.

Training loss

- `BCEWithLogitsLoss` with **label smoothing** (0.001): $y(1-\epsilon) + 0.5\epsilon$
- Model selection **always** uses the plain, non-smoothed log loss

- **AMP** (`torch.amp.GradScaler`) for speed and stability
- **Early stopping** (patience = 8 epochs) and **ReduceLROnPlateau**, both on the inner split
- Seeds set *before* the model is built: initial weights depend on the seed
- **In-RAM checkpointing**: best `state_dict` cloned to CPU, written to disk once at the end
- Explicit `torch.cuda.empty_cache()` so sibling workers reclaim VRAM

Why it matters

Avoids dozens of slow disk writes per fold and prevents GPU memory creep across 30 jobs.

Validation Protocol

Outer loop: 5-fold OOF

`MultilabelStratifiedKFold`, seed 42, on treated rows. Each fold's validation part yields the out-of-fold predictions and is scored **once**, after training.

Inner split: model selection

10% of each training fold (stratified) drives early stopping, LR scheduling and best-epoch choice of the neural networks. CatBoost uses fixed iterations.

Blend: nested CV

Weights are fitted on $k-1$ folds' OOF predictions and scored on the held-out fold.

Known limitations

No drug identifier: the CV cannot be grouped by compound, so the same compound may appear in training and validation. Design choices were also made by looking at these same folds. Scores are optimistic (see the Kaggle test-set slide).

NN Cross-Validation at Scale

- 2 networks $\times$ 3 seeds $\times$ 5 folds $=$ **30 jobs**
- `joblib` + `loky` backend, GPUs assigned **round-robin**
- 2 GPUs $\times$ 2 jobs/GPU $=$ **4 parallel workers**
- Predictions averaged over the 3 seeds per (fold, model)
- Wall-clock: NN CV 95 min, CatBoost CV 29 min, inference 0.5 min (total $\approx$130 min)

```
NN_JOBS_PER_GPU = 2
N_JOBS_NN = max(1, N_GPUS * NN_JOBS_PER_GPU)

Parallel(n_jobs=N_JOBS_NN,
       backend="loky")(
 delayed(_train_nn_job)(...)
 for job in jobs
)
```

Ensemble Blending: Method and Nested-CV Results

Weight search

- Softmax-reparameterised weights on the simplex
- **Nelder–Mead**, not SLSQP: avoids a zero finite-difference gradient stalling the optimiser at x_0

Nested CV

- Weights fit on $k-1$ folds, scored on the held-out fold; final weights = mean across folds
- In-sample score (weights fit on all OOF rows): 0.01606, the nested one is 0.01607; gap $1.58 \cdot 10^{-6}$. The blending step is not optimistic with three weights; this says nothing about leakage from the CV design

Score and weights per held-out fold

Fold	Score	w_{resnet}	w_{ft}	w_{cb}
0	0.01605	0.45	0.377	0.173
1	0.01618	0.468	0.367	0.165
2	0.01575	0.431	0.387	0.182
3	0.01612	0.448	0.37	0.181
4	0.01623	0.468	0.372	0.16

Final weights (mean across folds)

ResNet 45.29% | FT-Transformer 37.46% | CatBoost 17.25%

Honest nested-blend score

0.01607 ± 0.00017

Results – Cross-Validation

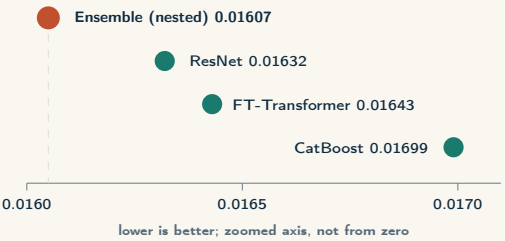
Validation loss per fold

Fold	ResNet	FT-Tr.	CatBoost
0	0.01686	0.01693	0.01707
1	0.01705	0.01714	0.01707*
2	0.01652	0.01657	0.01676
3	0.01679	0.01690	0.01701*
4	0.01700	0.01702	0.01704
Mean	0.01684	0.01691	0.01699

* 1 constant target in the training fold, predicted with the smoothed prior.

Network values are the *mean of the per-seed losses*; CatBoost is trained once per fold. ResNet is the best model in every fold.

Out-of-fold log loss (seed-averaged)



Seed-averaged predictions lower the loss by about 0.0005 per network (bagging gain). Fold standard deviation of the blend: 0.00017. Ensemble vs. best single model: about 1.5% better.

Independent Check: Kaggle Test Set

Estimate	Log loss
Nested CV, treated rows only	0.01607 ± 0.00017
Rescaled to all rows ($\times$ 0.910)	0.01462
Kaggle private	**0.01634** (+11.8%)
Kaggle public	0.01865 (+27.5%)

CV scores use treated rows only. Control rows are predicted 0 (true 0) and add almost no loss, so the all-rows score $\approx$ treated-only score $\times$ treated share (0.910 in the public test set). Percentages are relative to the rescaled estimate.

- **Late submission** through a separate **inference notebook**: loads 30 checkpoints, 5 CatBoost models, 10 fold pipelines and the blend weights; no training, no internet
- Submitted **once**; the test set was never used for tuning

The CV estimate is optimistic

The private score is 11.8% above the rescaled CV estimate. A likely cause is the missing grouping by compound (same compound in training and validation, different compounds in the test set) – **not tested**. Public and private differ by 0.0023, over ten times the fold standard deviation: the fold std understates the uncertainty on unseen data.

Ablation – Is an Extra Network Worth It?

Base: ResNet + FT + CatBoost	Extra OOF	Ensemble (nested)	Extra weight
+ MLP (PCA features)	0.01649	0.01604	1.5%
+ 1D-CNN (PCA features)	0.01661	0.01605	8.2%
+ 1D-CNN (raw features)	0.01661	0.01606	7.0%
no extra network (baseline at the time)	–	0.01606	–
no extra network (final rerun)	–	**0.01605**	–

Reading

All ensembles score 0.01604–0.01606: the differences ($\leq 2 \cdot 10^{-5}$) are below the fold standard deviation (0.00017) and the run-to-run GPU noise ($\approx 10^{-5}$). The ablation ran on an earlier code revision.

Decision

The extra network earns weight but no measurable gain, even with raw features. The simpler 3-model blend is kept: fewer components to train, explain and maintain.

Conclusions, Limitations and Future Work

Findings

- Ensemble 0.01607 vs. best single model (ResNet) 0.01632: $\approx$1.6% better
- ResNet and FT-Transformer carry $\approx$83% of the weight; CatBoost, weakest alone, gets $\approx$17%
- An extra network (MLP or 1D-CNN) does not improve the blend
- Kaggle test set: private 0.01634, public 0.01865, against a rescaled CV estimate of 0.01462

Future work

- Group the CV by compound, or reserve a hold-out before tuning
- Ablations: PCA vs. raw for the ResNet

Limitations

- No compound identifier: possible leakage across folds; the CV estimate is optimistic (confirmed by the Kaggle gap, cause not tested)
- Design choices made on the same folds that are reported; no hold-out reserved before tuning
- The inner split removes 10% of the training data; an earlier protocol that selected epochs on the OOF fold gave 0.01595, a difference consistent with selection optimism but mixed with the data loss and noise
- GPU non-determinism: repeated runs differ by $\approx 10^{-5}$

References

- Kaggle, *Mechanisms of Action (MoA) Prediction*, 2020. https://www.kaggle.com/competitions/lish-moa
- Gorishniy et al., *Revisiting Deep Learning Models for Tabular Data*, NeurIPS 2021 (FT-Transformer).
- Prokhorenkova et al., *CatBoost: unbiased boosting with categorical features*, NeurIPS 2018.
- Sechidis, Tsoumakas, Vlahavas, *On the Stratification of Multi-Label Data*, ECML PKDD 2011.
- Bradberry, *iterative-stratification* (Python package). https://github.com/trent-b/iterative-stratification
- Paszke et al., *PyTorch*, NeurIPS 2019; Pedregosa et al., *Scikit-learn*, JMLR 12, 2011.
- Loshchilov, Hutter, *Decoupled Weight Decay Regularization* (AdamW), ICLR 2019.
- Micikevicius et al., *Mixed Precision Training*, ICLR 2018.
- Szegedy et al., *Rethinking the Inception Architecture for Computer Vision* (label smoothing), CVPR 2016.
- Nelder, Mead, *A simplex method for function minimization*, The Computer Journal 7(4), 1965.